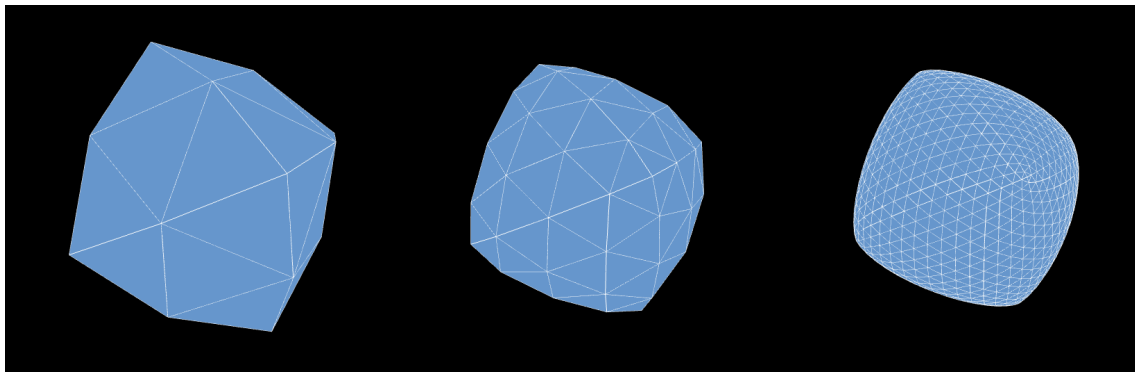


Visual Computing

Exercise 11: Curves & Surfaces

Hand-out Date: 9 December 2025



Goals

- Understand the concept of B-Spline curves and the de Boor algorithm.
- Understand the principles of mesh subdivision.

Resources

The lecture slides and exercise slides are accessible via the [Visual Computing Course Web Page](#).

Tasks

1. B-spline Curve

In this task, we will construct the quadratic B-Spline step by step to understand its concept. Recall that B-spline curves are defined by their degree, a set of control points, and a knot vector. The control points influence the shape of the curve. The knot vector is a sequence of values that determines the parameterization of the B-spline curve. They divide the curve into segments, each associated with a specific set of control points that influence it. Suppose we are given four de Boor points $\mathbf{p}_0$, $\mathbf{p}_1$, $\mathbf{p}_2$, $\mathbf{p}_3$ as seen in Figure 1. From the lecture and script, you already know that a B-spline of degree $n = 2$ and with $k + 1 = 4$ de Boor points consequently requires $n + k + 2 = 7$ knots in its knot vector. We therefore define u_0 , u_1 , u_2 , u_3 ,

u_4, u_5, u_6 . Further, let's assume that we deal with a Cardinal (uniformly spaced knots) *unclamped* (not interpolating the end points) B-spline with the following knot relations: $u_0 < u_1 < u_2 < u_3 < u_4 < u_5 < u_6$. How would the knot vector look like if we wanted a clamped B-spline? What if we were to add or remove control points in the example? What changes would we need to make to the knot vector, and how would the curve change?

In the lecture, we saw two ways of evaluating B-splines. The more principled Cox-de Boor formulation, which defines the B-spline as the sum of all polynomial bases, and the de Boor algorithm, which, similarly to the de Casteljau algorithm, recursively builds new de Boor points of higher order. In the following, we'll explore another ground-up approach to building B-splines to build some more intuition. In the end, we will once again arrive at the de Boor formulation for B-splines seen in the lecture. Also, remember that $\mathbf{d}_i^0 = \mathbf{p}_i$ per definition.

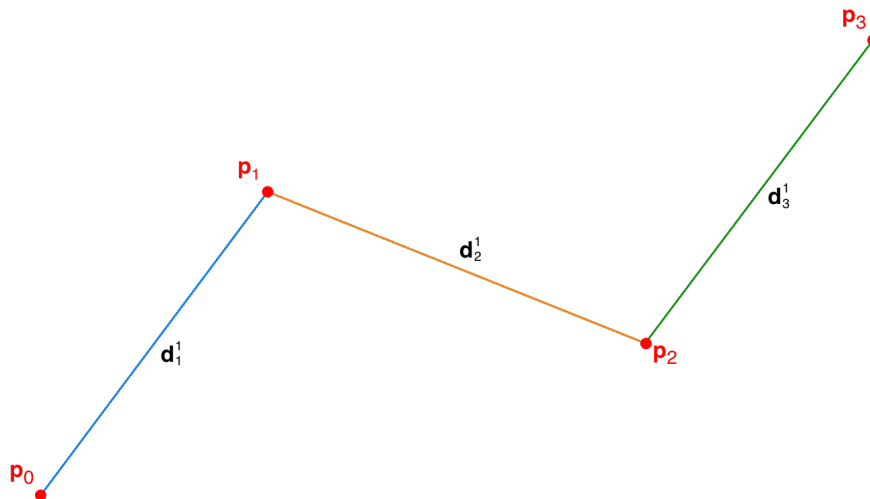


Figure 1: Linear B-spline.

Linear B-spline. We first build linear B-spline segments of degree 1 between consecutive de Boor points. Given two points $\mathbf{d}_0^0 = \mathbf{p}_0$ and $\mathbf{d}_1^0 = \mathbf{p}_1$, as well as the knots u_0, u_1, u_2 and u_3 (remember that for $k+1 = 2$ control points and degree $n = 1$, we require $n+k+2 = 4$ knots), the convex combination of these two points gives a line segment as

$$\mathbf{d}_1^1(t \mid \mathbf{d}_0^0, \mathbf{d}_1^0; u_1, u_2) = \frac{u_2 - t}{u_2 - u_1} \mathbf{d}_0^0 + \frac{t - u_1}{u_2 - u_1} \mathbf{d}_1^0, \quad t \in [u_1, u_2],$$

where the superscript 1 means the degree is one. Note that we omit the formulas for the boundaries between u_0 and u_1 , as well as between u_2 and u_3 , since the linear B-spline is only fully defined for $t \in [u_n, u_{k+1}]$, i.e. $[u_1, u_2]$ since $n = 1$ and $k + 1 = 2$ for the linear B-spline segment, as seen in the lecture. However, u_0

and u_3 are still necessary to guarantee the boundary behavior and differentiability of the linear B-spline. Write down the formulas for the other 2 consecutive line segments $\mathbf{d}_2^1$ and $\mathbf{d}_3^1$, and create the formula for the complete linear B-spline made up of these line segments. Make sure that the individual segments meet at suitable knots. You only need to write up the valid parameter ranges, and you can omit the boundary formulas. *Additional question: How many knots are necessary for this linear B-spline in total? How does it compare to the number of knots necessary for the B-spline of degree 2 that we are building?*

This construction can now be generalized to produce smooth, piecewise-polynomial curves of higher degree in the next step.

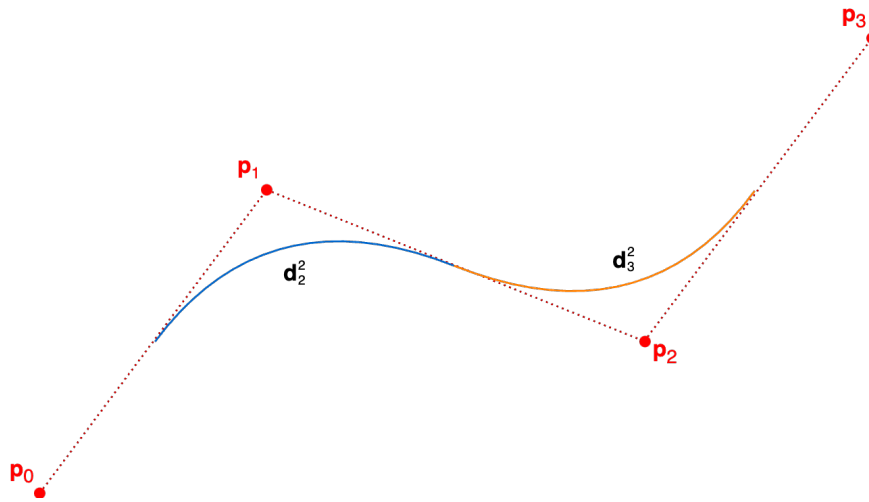


Figure 2: Quadratic B-spline.

Quadratic B-spline. Going to one degree higher, the construction of quadratic spline curves is based on the repeated averaging of two consecutive line segments. For two consecutive line segments $\mathbf{d}_1^1$ and $\mathbf{d}_2^1$, we can define the quadratic B-Spline curve as

$$\mathbf{d}_2^2(t \mid \mathbf{d}_0^0, \mathbf{d}_1^0, \mathbf{d}_2^0, \mathbf{d}_1^1, \mathbf{d}_2^1; u_1, u_2, u_3, u_4) = \frac{u_3 - t}{u_3 - u_2} \mathbf{d}_1^1(t \mid \mathbf{d}_0^0, \mathbf{d}_1^0; u_1, u_3) + \frac{t - u_2}{u_3 - u_2} \mathbf{d}_2^1(t \mid \mathbf{d}_1^0, \mathbf{d}_2^0; u_2, u_4) \quad t \in [u_2, u_3].$$

Here, the reason for reparameterizing the line segments is that the interval $[u_2, u_3]$ is then within the domain of both segments. This ensures that the two line segments can be combined in a *convex* combination to form a quadratic curve segment. Again, we left out the boundary segments and concentrated on the valid, overlapping parameter range. Write down the formula for the next quadratic curve segment in the same fashion, and create the formula for the quadratic B-spline comprised of these two curve segments. Again, make sure that they meet at a suitable knot. Finally,

you can compare the resulting formula with the de Boor algorithm seen in the lecture and script and convince yourself that we end up at the identical formulation. *Additional question: How would we go about building a B-spline of degree 3 out of the two B-splines of degree 2? What would be the overlapping knot sequence there?*

Properties. Given the construction process shown above, you may now have a deeper understanding of the B-spline curve. Write down four properties of the B-spline curve.

Implement de Boor's algorithm As previously mentioned, this construction process can be summarized by de Boor's algorithm. You can try to implement the algorithm using any software that you are familiar with. This helps to understand how changes in these elements impact the curve. A Python script `deBoor.py` is provided that includes a minimal implementation and visualization. The de Boor/control points are global parameters in the code, which you can modify as you like. You can run the script with the following command

```
python3 deBoor.py --degree 2
```

When the first and last knots have multiplicity of degree + 1, the B-Spline curve interpolates the endpoints. You can see the visualization by running:

```
python3 deBoor.py --degree 2 --interpolate_endpoints
```

2. Mesh Subdivision (Loop algorithm)

In this task, you will concentrate on implementing the Loop subdivision algorithm. Loop subdivision is a method for refining triangular meshes to produce smoother surfaces. Your task is to apply the Loop subdivision algorithm to a given triangular mesh structure. You will start with the existing code framework, and complete the missing part. Key aspects of this task include:

- (a) **Wireframe Rendering:** Wireframe is often used for visualization when it comes to geometry processing. Find the relevant code of **Task 2a** in the code and implement the missing lines. A custom mesh class is already implemented for you, and you can retrieve wireframe vertices using `getWireframeVertices`. In `render` function, bind the corresponding buffers and render the line geometry with `gl.drawArrays(gl.LINES, ...)`.
- (b) **Helper Functions to Compute Adjacency:** Adjacency information is necessary for all subdivision algorithms. Read the code of `Mesh`, `Face`, `Edge` and `Vertex` class and understand how the mesh structure is stored in such data structures. Fill in the function `sfaceAdjacency` and `vertexAdjacency`.

- (c) **Implement Loop Algorithm:** Implement Loop algorithm in `LoopSubdivision`. You can refer to the C++ implement from [libigl](#), which is a header-only C++ geometry processing library. Note that the weights to compute new vertex positions are slightly different from the lecture slides. You can play with different number of subdivision iterations to see how the mesh evolves.